

13. Header Linked List – Time Complexity and Space Complexity

1. Creation of Header List:

- ***Time Complexity:***

→ Best = Worst = Average = $O(1)$.

- ***Space Complexity:***

Input Space = $O(1)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: Input Space + Auxiliary = $O(1) + O(1) = O(1)$.

2. Insert At Beginning:

- ***Time Complexity:***

→ Best = Worst = Average = $O(1)$.

- ***Space Complexity:***

→ Considering only int data as input, then:

Input Space = $O(1)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: Input Space + Auxiliary = $O(1) + O(1) = O(1)$.

→ Considering `n` element as input, then:

Input Space = $O(n)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: Input Space + Auxiliary = $O(n) + O(1) = O(n)$.

3. Insert At END:

- ***Time Complexity:***

→ ***Best = $\Omega(1)$. $\left[\begin{array}{l} \text{When List is Empty} \\ \text{and we enter first data to list.} \end{array} \right]$***

→ ***Worst Case = $O(n)$ = Average Case = $\Theta(n)$.
 $\left[\begin{array}{l} \text{When List is not empty and there is numerous} \\ \text{Nodes in the list. Note: True Average Case doesn't} \\ \text{exist here.} \end{array} \right]$***

- ***Space Complexity:***

→ ***Considering only int data as input, then:***

***Input Space = $O(1)$.
Auxiliary Space = $O(1)$.***

Total Space Complexity: Input Space + Auxiliary = $O(1) + O(1) = O(1)$.

→ ***Considering `n` element as input , then:***

***Input Space = $O(n)$.
Auxiliary Space = $O(1)$.***

Total Space Complexity: Input Space + Auxiliary = $O(n) + O(1) = O(n)$.

3. Insert At Position:

- **Time Complexity:**

→ **Best** = $\Omega(1)$. [When $Pos < 1$ or $Pos > head \rightarrow data + 1$]

→ **Worst Case** = $O(n)$. [When Position is not found.]

→ **Average Case**

[Based on Loop]

→ If the position is 1, the loop's inner statement runs 0 time.

→ If the position is 2, the loop's inner statement runs 1 time.

→ If the position is 3, the loop's inner statement runs 2 times.

→ If the position is 4, the loop's inner statement runs 3 times.

.

.

→ If the position is $(N + 1)$, the loop's inner statement runs : N times ,

As when $i = N < N$ becomes false , hence no execution of loop's inner statements.

$$\text{Hence, } \sum_{i=1}^N i = \frac{N(N+1)}{2}$$

Now, as we take all the possibilities then , we take the best case also i.e.

$O(1)$, when position = 1 . Considering ' $T(P)$ ' is time complexity of position ' P ' .

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ P - 1 & , \text{when } P = 2, 3, \dots, N + 1. \end{cases}$$

Now, If there is `N` no. of nodes there we can insert data at $N + 1$ positions.

For position 1, the probability of insertion is $\frac{1}{N + 1}$.

For position 2, the probability of insertion is $\frac{1}{N + 1}$.

.

.

For position $N + 1$, the probability of insertion is $\frac{1}{N + 1}$.

Hence probability of insertion will always remain $\frac{1}{N + 1}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N + 1} \times 1 + \frac{1}{N + 1} \times 2 + \dots + \frac{1}{N + 1} \times N \right) + \frac{1}{N + 1} \times 1$$

$$= \frac{1}{N + 1} \times \{(1 + 2 + 3 + \dots + N) + 1\}$$

$$= \frac{1}{N+1} \times \left\{ \left(\sum_{i=1}^N i \right) + 1 \right\}$$

$$= \frac{1}{N+1} \times \left\{ \frac{N(N+1)}{2} + 1 \right\}$$

$$= \left\{ \frac{N(N+1)}{2(N+1)} + \frac{1}{N+1} \right\}$$

$$= \left\{ \frac{N}{2} + \frac{1}{N+1} \right\}$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant $\frac{1}{2}$, we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms, largest growth rate is the result:

→ $\Theta(N)$ grows linearly with N .

→ $\Theta\left(\frac{1}{N}\right)$ decreases to a constant, i. e. when N grows larger, $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n} \right) \Rightarrow \frac{1}{\infty} = 0 \quad \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant} \right].$$

As $\Theta(N)$'s growth rate is higher , therefore , average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

*Average Case Time Complexity[Most Simplest Alternative Apporach]:
(Simplest approach)*

For position 1, the probability of insertion is $\frac{1}{N+1}$.

For position 2, the probability of insertion is $\frac{1}{N+1}$.

.

.

For position $N+1$, the probability of insertion is $\frac{1}{N+1}$.

*Hence probability of insertion will always remain $\frac{1}{N+1}$,
for position 1 to $N+1$.*

Average case complexity : Probability of insertion $\times$ $\left(\begin{array}{c} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array} \right)$

$$= \frac{1}{N+1} \times \left(\sum_{i=1}^{N+1} i \right)$$

$$= \frac{1}{N+1} \times \left(\frac{1}{2} (N+1)((N+1)+1) \right) \left[\text{As, } \sum_{k=1}^n k = \frac{1}{2} (n)(n+1) \right]$$

$$= \frac{1}{N+1} \times \left(\frac{(N+1)(N+2)}{2} \right)$$

$$= \frac{N + 2}{2}$$

$$= \frac{N}{2} + \frac{2}{2}$$

$$= \frac{N}{2} + 1$$

Applying Big Theta Θ we get:

$$= \Theta\left(\frac{N}{2}\right) + \Theta(1)$$

$$= \frac{1}{2} \times \Theta(N)$$

$$= \Theta(N)$$

Hence, Average Time Complexity is : $\Theta\left(\frac{N}{2} + 1\right)$ or $\Theta(N)$.

- **Space Complexity:**

→ Considering only int data, int pos as input, then:

Input Space = $O(1)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: Input Space + Auxiliary = $O(1) + O(1) = O(1)$.

→ Considering `n` element as input i.e. `n` nodes , then:

Input Space = $O(n)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: Input Space + Auxiliary = $O(n) + O(1) = O(n)$.

3.Insert After Element:

- **Time Complexity:**

$$\rightarrow \text{Best} = \Omega(1). \left[\begin{array}{l} \text{When List is Empty} \\ \text{and Head} \rightarrow \text{Next} = \text{NULL} \end{array} \right]$$

$$\rightarrow \text{Worst Case} = O(n). \left[\begin{array}{l} \text{When Element is not found} \\ \text{and List is not empty i. e.} \\ \text{Head} \rightarrow \text{Next} \neq \text{NULL} \end{array} \right]$$

$\rightarrow$ **Average Case**

$\rightarrow$ If the Element is inserted after 1st node, the loop's inner statement runs 0 time.

$\rightarrow$ If the Element is inserted after 2nd node, the loop's inner statement runs 1 time.

$\rightarrow$ If the Element is inserted after 3rd node, the loop's inner statement runs 2 time.

$\rightarrow$ If the Element is inserted after 4th node, the loop's inner statement runs 3 time.

.

.

$\rightarrow$ If the Element is inserted at $(n + 1)^{\text{th}}$ node, the loop's inner statement runs $N - 1$ time.

$$\therefore, \sum_{i=0}^{N-1} i = \frac{N(N-1)}{2}$$

Now, If there is 'N' no. of nodes there we can insert data at 2 to N + 1 positions or repositioning 1 to N .

For position 1, the probability of insertion is $\frac{1}{N}$.

For position 2, the probability of insertion is $\frac{1}{N}$.

.

.

For position $N + 1$, the probability of insertion is $\frac{1}{N}$.

Hence probability of insertion will always remain $\frac{1}{N}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations)

$$= \frac{1}{N} \times \left(\sum_{i=0}^{N-1} i \right)$$

$$= \frac{1}{N} \times \frac{N(N-1)}{2}$$

$$= \frac{N-1}{2}$$

Applying , Big O on the above equation:

$$= \frac{1}{2} \times O(N) - O\left(\frac{1}{2}\right)$$

$$= O(N)$$

Hence, Average Time Complexity is : $\Theta(N)$.

Average Case Time Complexity[Most Simplest Alternative Approach]:
(Simplest approach)

***(If the elements in the linked list are uniformly distributed
and the key is equally likely to be any element in the list)***

***Note: –The checking that Element or Key is found or not , done in the
condition of while loop, and the condition checks `n` times .***

–if key is found at 1st Node , Key/Element searched for 1 time .

–if key is found at 2nd Node , Key/Element searched for 2 time .

–if key is found at 3rd Node, Key/Element searched for 3 time .

.

.

–if key is found at Nth Node , Key/Element searched for N time .

As , the key is equally likely to be any element in the list,

Probability of finding 1st element (p_1), if key equals to 1st element = $\frac{1}{N}$.

Probability of finding 2nd element (p_2), if key equals to 2nd element = $\frac{1}{N}$.

.

.

Probability of finding Nth element (p_n), if key equals to Nth element = $\frac{1}{N}$.

Therefore,

Probability(P) = $p_1 = p_2 = \dots = p_n = \frac{1}{N}$.

Therefore, Average Case = Number of times the Key is searched × probability of finding elements.

That is the average number of executions is the sum of all possible executions weighted by their probabilities. This is given by:

$$\begin{aligned}\therefore \text{Average Case} &= \frac{1}{N} \times \left(\sum_{i=1}^N i \right) \\ &= \frac{1}{N} \times \frac{N(N+1)}{2} \left[\sum_{i=1}^N i = \frac{N(N+1)}{2} \right] \\ &= \frac{N+1}{2} \\ &= \frac{N}{2} + \frac{1}{2}\end{aligned}$$

Applying , Big Theta Θ we get :

$$\begin{aligned}&= \frac{1}{2} \times (\Theta(N) + \Theta(1)) \\ &= \Theta(N) + \Theta(1) \\ &= \Theta(N).\end{aligned}$$

- ***Space Complexity:***

→ ***Considering only int data, int element as input, then:***

Input Space = $O(1)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: $Input\ Space + Auxiliary = O(1) + O(1) = O(1)$.

→ ***Considering `n` element as input i.e. `n` nodes , then:***

Input Space = $O(n)$.

Auxiliary Space = $O(1)$.

Total Space Complexity: $Input\ Space + Auxiliary = O(n) + O(1) = O(n)$.
